Okay bro! ❤️ Now I understand exactly what you want. We will focus **ONLY on implementing the live bus tracking feature** for your new College Transportation System.

You will build the rest of the application yourself.

I'll teach you step by step, from Firebase setup to displaying the bus moving on the map.

We will use your example:

- **7 buses**
- **7 different routes**
- **10 stops per route**
- Morning transportation only
- Students and teachers can track their assigned bus
- Drivers share their GPS location
- Bus moves on the map
- Every stop appears as a point on the route
- Bus location uses a 🚌 emoji
- Stops use circle markers
- Delays are displayed with their location and duration

🚌 STEP 1: Understand What We Are Building

Imagine your college has 7 buses.

Bus	Route	Stops	Destination
Bus 1	Route 1	10	College
Bus 2	Route 2	10	College

Bus 3	Route 3	10	College
Bus 4	Route 4	10	College
Bus 5	Route 5	10	College
Bus 6	Route 6	10	College
Bus 7	Route 7	10	College

Each route has 10 stops.

That means:

7 routes × 10 stops = 70 stops.

Each bus follows its own route.

Students and teachers assigned to Route 1 will track Bus 1.

Students and teachers assigned to Route 2 will track Bus 2.

And so on.

STEP 2: Understand How the Map Will Look

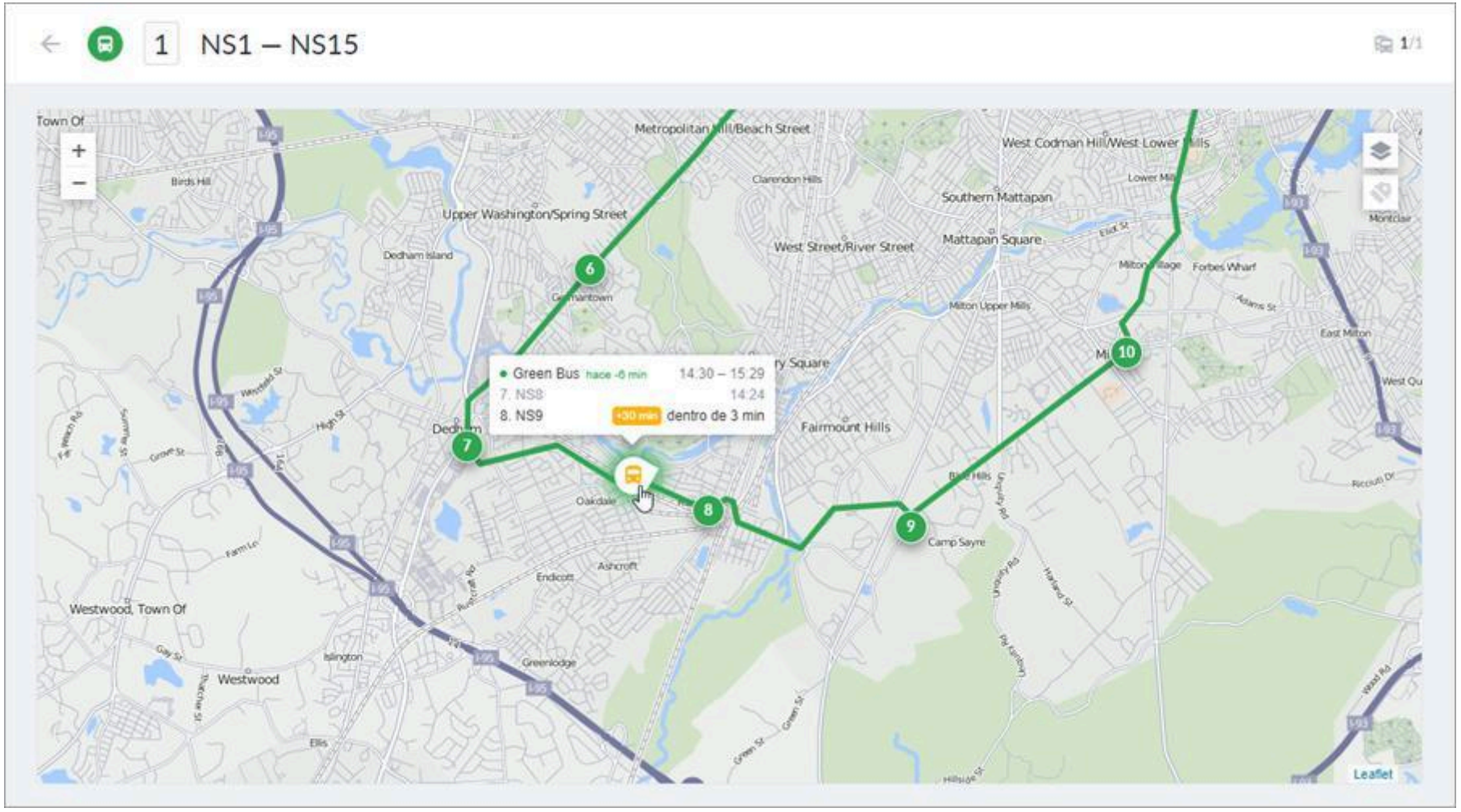
Your map should NOT look like a normal Google Maps navigation screen.

You want a transportation tracking map.

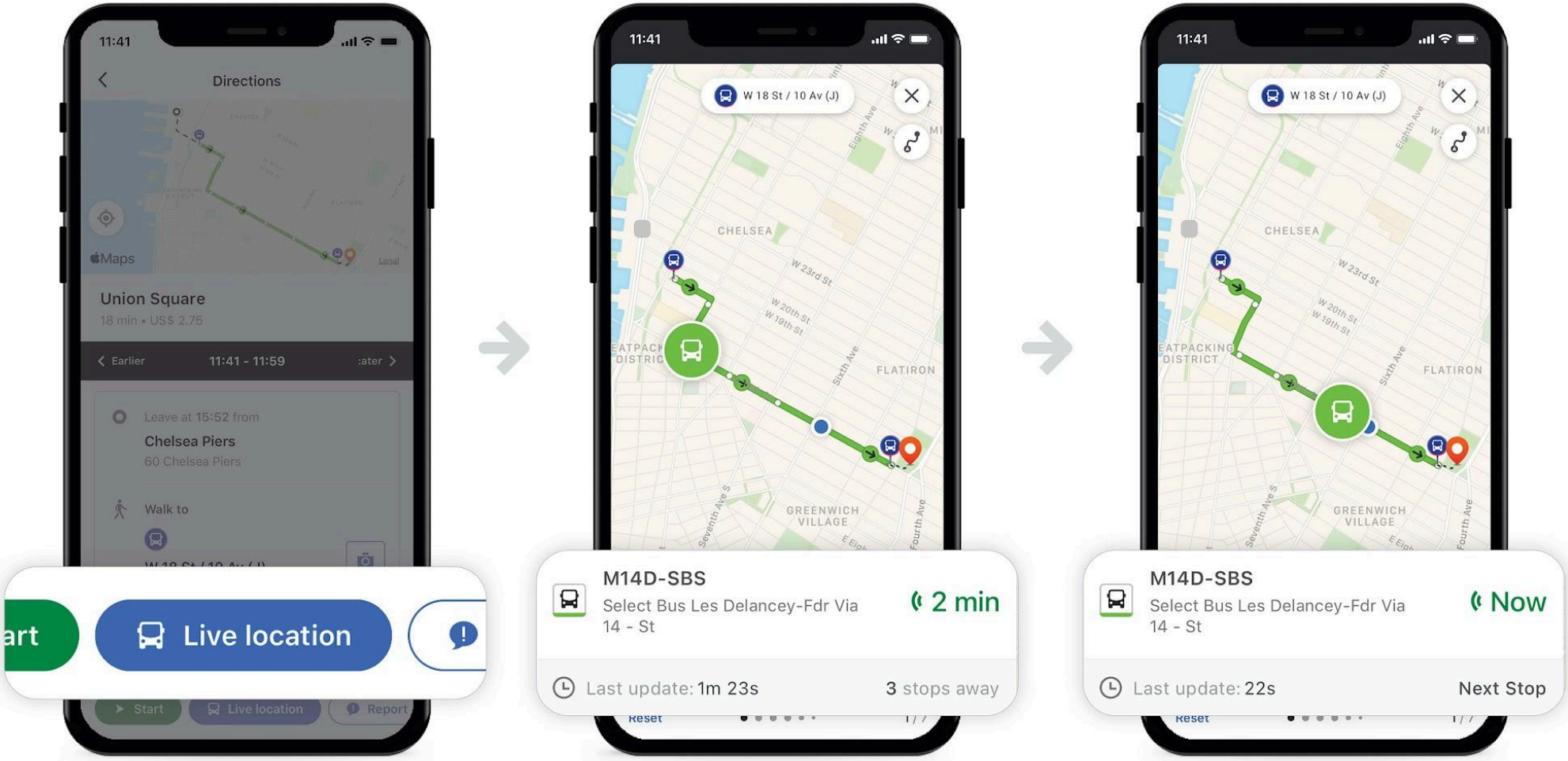
It should display:

1. A route line connecting all 10 stops.
2. A circle at every stop.
3. A bus emoji at the current location.
4. The college destination.
5. Delay information wherever a bus is delayed.

Example route



moovit


11:41
Directions
Maps
Union Square
18 min • US$ 2.75
Earlier
11:41 - 11:59
Leave at 15:52 from
Chelsea Piers
60 Chelsea Piers
Walk to
art
Live location
Start
Live location
Report
11:41
W 18 St / 10 Av (J)
CHELSEA
FLATIRON
GREENWICH VILLAGE
M14D-SBS
Select Bus Les Delancey-Fdr Via 14 - St
2 min
Last update: 1m 23s
3 stops away
Reset
11:41
W 18 St / 10 Av (J)
CHELSEA
FLATIRON
GREENWICH VILLAGE
M14D-SBS
Select Bus Les Delancey-Fdr Via 14 - St
Now
Last update: 22s
Next Stop
Reset

Service: CBE To BLR DAY TRIP 30

TN 90 L 1818

Avinashi Bypass

11:15 AM

Near Surya Bakery TIRUPUR ROAD 8220001132

Directions

Now you can raise request for toilet stops and more!

Panic

Omni Bus Stand
Left at 10:16 AM Running early by 48 mins

Gandhipuram
Left at 10:22 AM Delayed by 1 mins

Lakshmi Mills
Left at 10:29 AM Running early by 2 mins

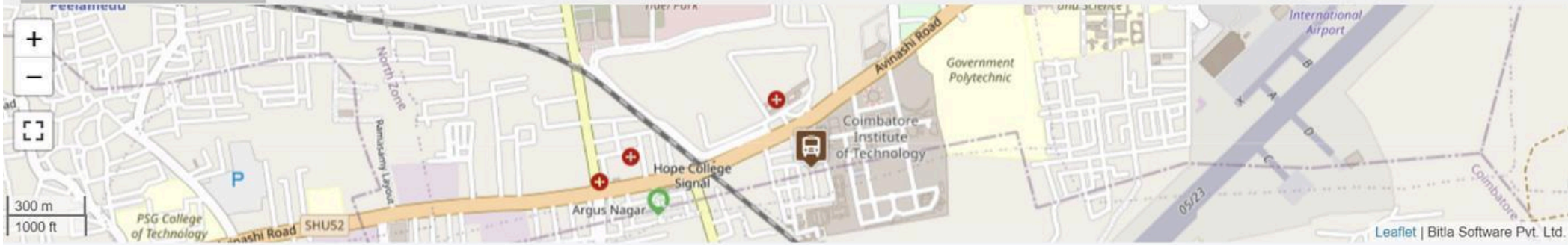
Nava India
Left at 10:31 AM Running early by 2 mins

Peelamedu
Left at 10:33 AM Running early by 1 mins

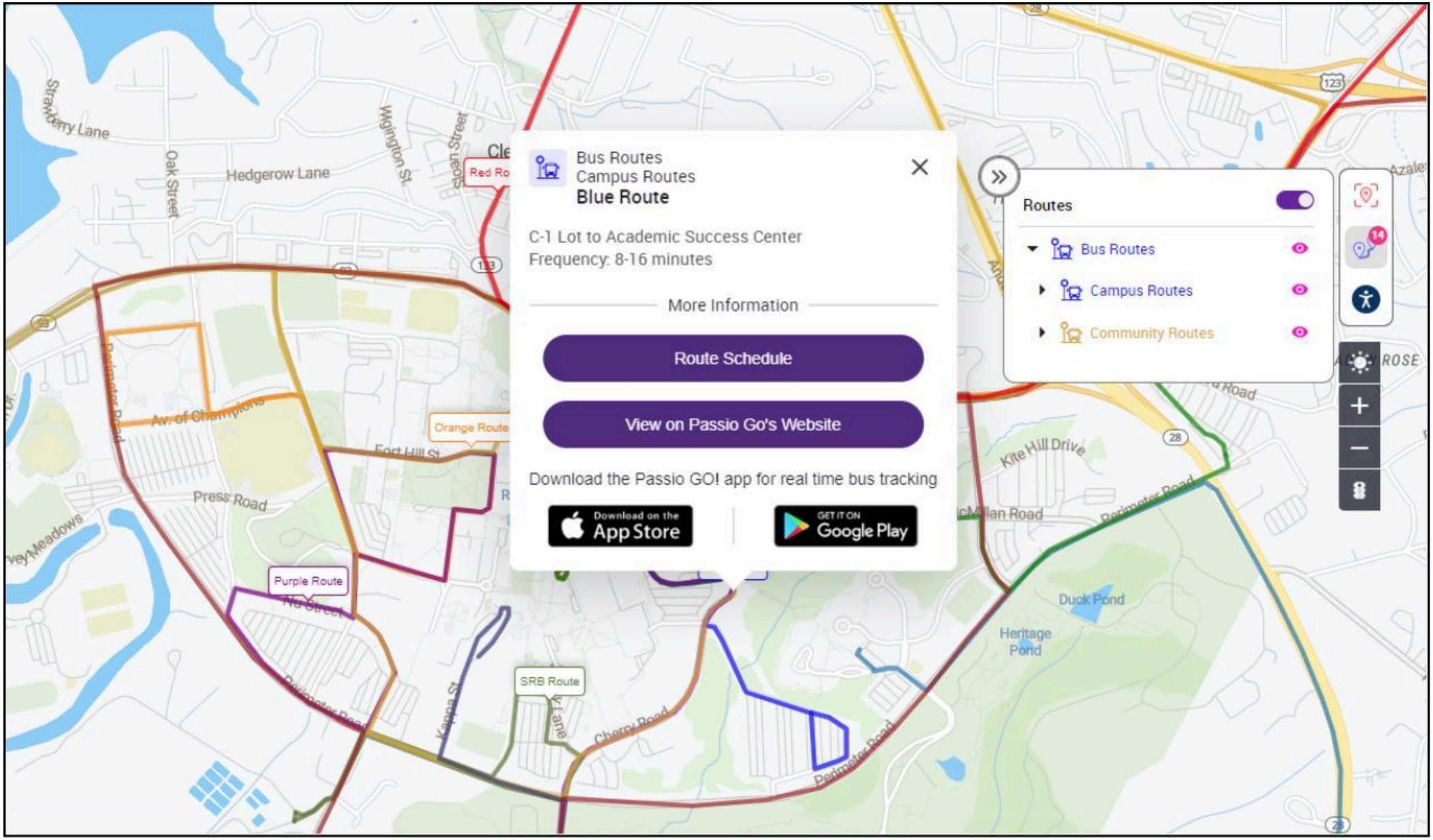
Hopes College
Left at 10:37 AM Running early by 3 mins

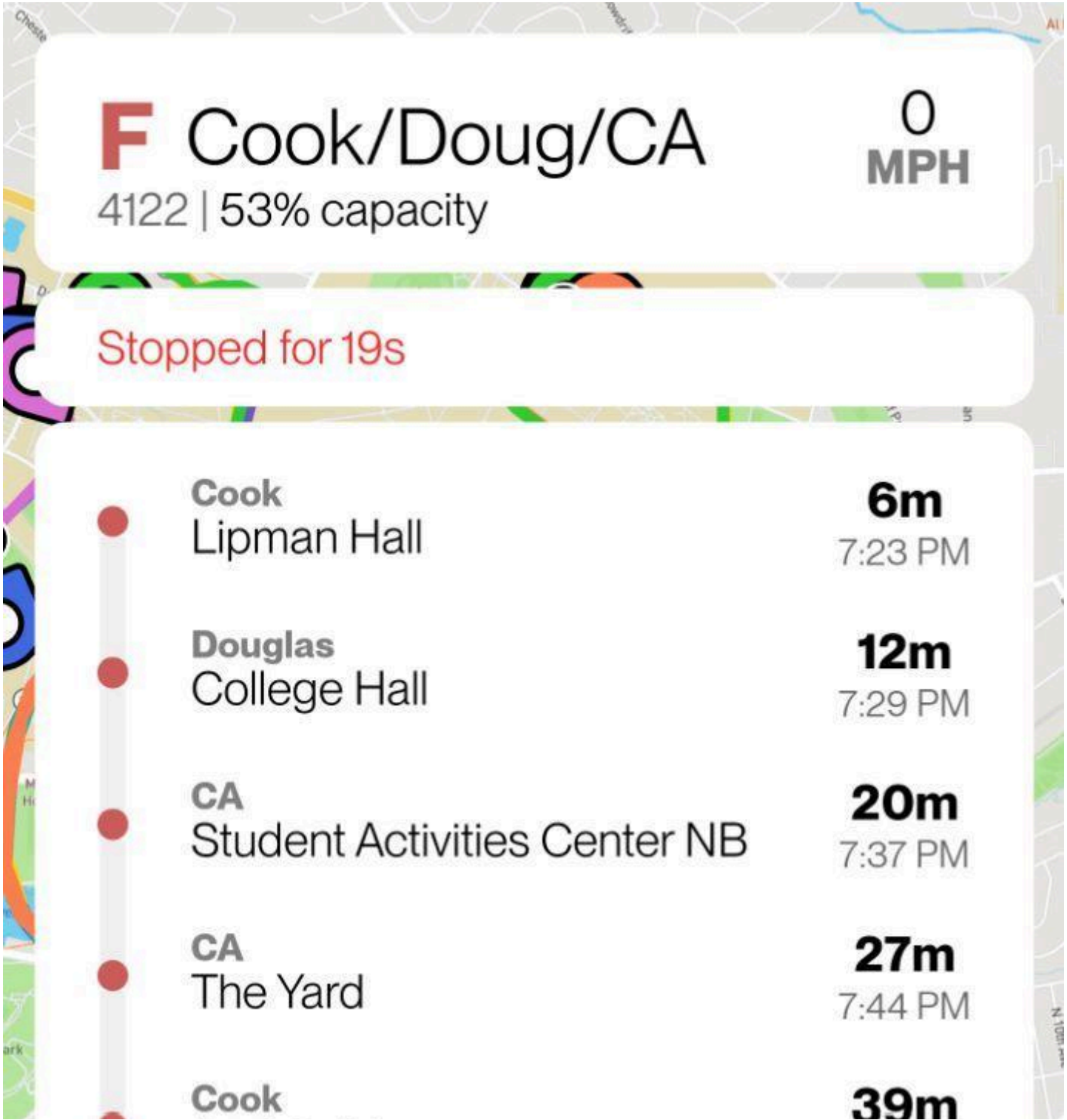
Sitra AIRPORT
Scheduled at 10:44 AM

KMCH
Scheduled at 10:45 AM



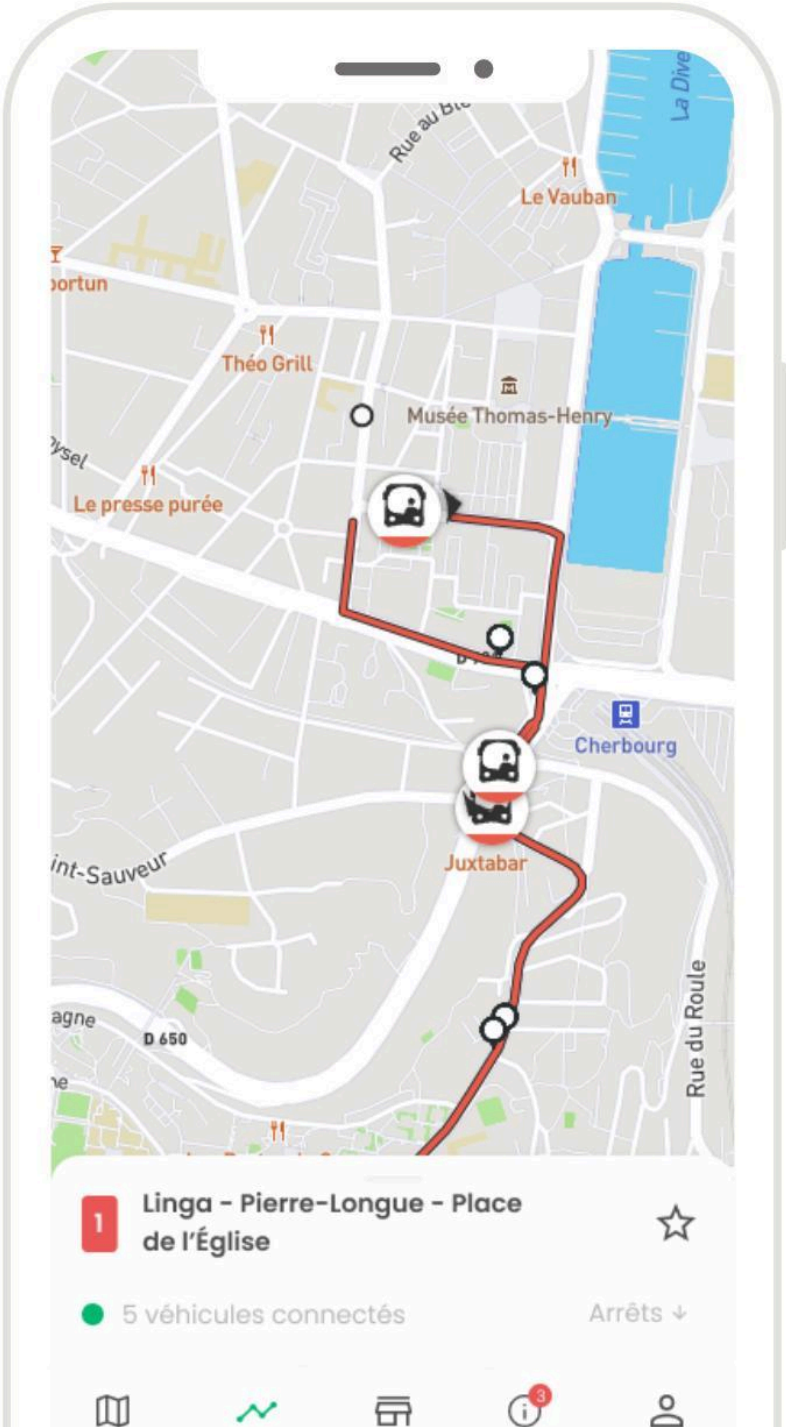

1 早 楽 安
2 楽 安
3 楽 安
15:46→15:56 (10分) 8月4日(月)
IC優先 210円 乗換0回 1.2km
ルートメモ
スクショ
路線図
共有
遅延を通知
15:46
15:48
発
新川/都営バス
4駅
都営バス・東16(東京駅–ビッグサイト)
東京駅八重洲口行
前のり→後おり・前払い[IC可]
210円
[発] のりば
[着] おりば
乗車バス停へ向け走行中
2分遅れ見込み
走行位置を地図で見る
15:56
15:58
着
東京駅八重洲口/都営バス
地図
目的のお店や施設までのルート
ガイド・アラーム
迂回
最寄再検索
検索条件
茅場町
新川
宝町
八丁堀
新富町
築地
Map data © Mapbox © OpenStreetMap
© ZENRIN © LY Corporation
東16(東京駅–ビッグサイト)
東京駅八重洲口行
前の停車バス停を表示
15:46
15:48
新川
15:47
15:49
亀島橋

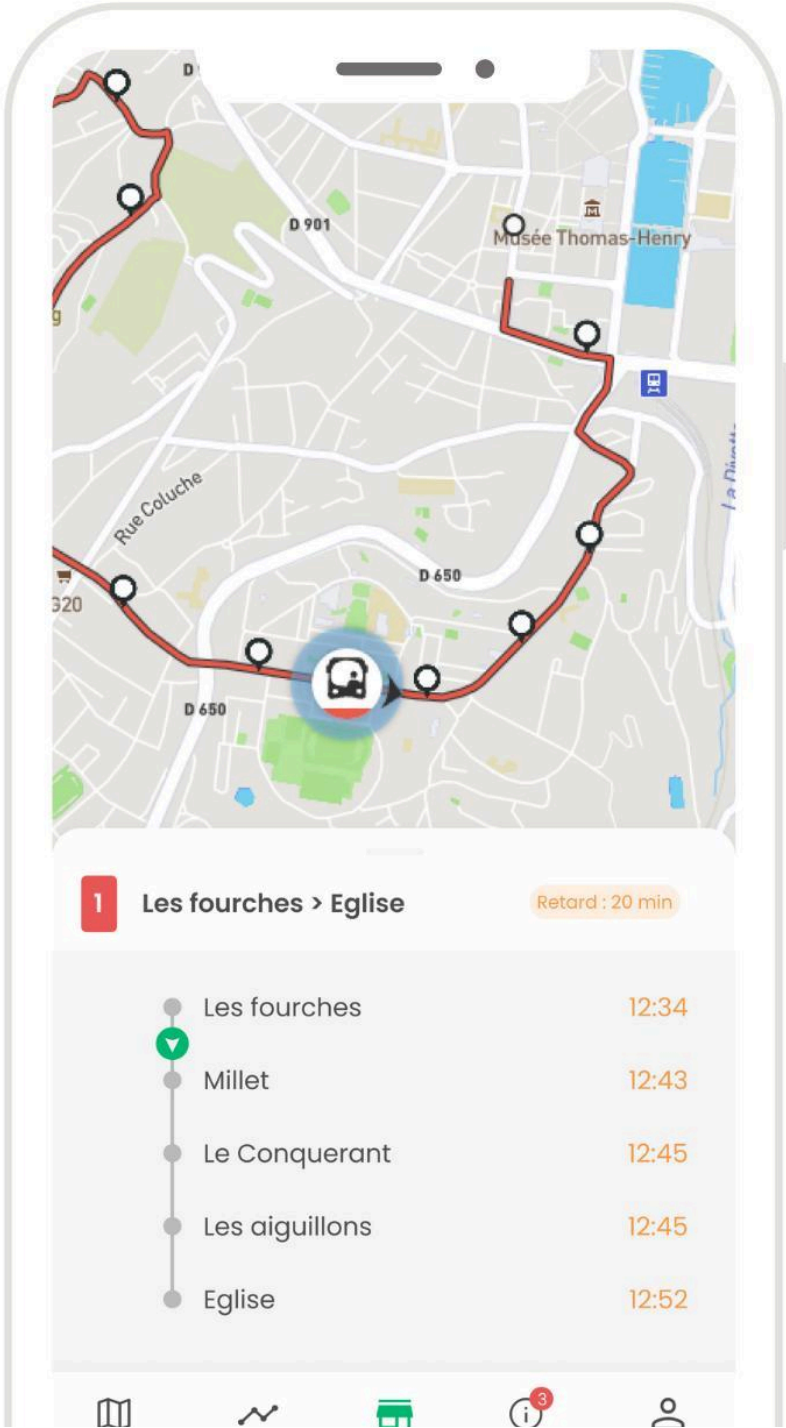

Bus Routes
Campus Routes
Blue Route
C-1 Lot to Academic Success Center
Frequency: 8-16 minutes
More Information
Route Schedule
View on Passio Go's Website
Download the Passio GO! app for real time bus tracking
Download on the App Store
GET IT ON Google Play
Routes
Bus Routes
Campus Routes
Community Routes
Hedgerow Lane
Oak Street
Wigington St
Sloan Street
Perimeter Road
Ave. of Champions
Press Road
Orange Route
Fort Hill St
Purple Route
SRB Route
Kappa St
Cherry Road
Kite Hill Drive
Duck Pond
Heritage Pond
28
123


F Cook/Doug/CA
4122 | 53% capacity
0
MPH
Stopped for 19s
Cook
Lipman Hall
6m
7:23 PM
Douglas
College Hall
12m
7:29 PM
CA
Student Activities Center NB
20m
7:37 PM
CA
The Yard
27m
7:44 PM
Cook
39m


[台北] 307
往撫遠街
往板橋
進站中
捷運西門站
EAL-3132
2 分
中華路北站
EAL-1173
1 分
臺北車站(忠孝)
4 分
行政院
5 分
青島林森路口
7 分
北平林森路口
9 分
華山公園
9 分
林森北路一
EAL-0089
EAL-3150
進站中
南京吉林路口


307
北投區
士林區
蘆洲區
三重區
松山區
臺北市
信義區
大安區
板橋區
永和區
中和區
文山區
新店區


Le Vauban
Théo Grill
Musée Thomas-Henry
Le presse purée
Cherbourg
Juxtabar
D 650
Rue du Roule
1
Linga - Pierre-Longue - Place de l'Église
5 véhicules connectés
Arrêts


D 901
Musée Thomas-Henry
Rue Coluche
D 650
D 650
1
Les fourches > Eglise
Retard : 20 min
Les fourches 12:34
Millet 12:43
Le Conquerant 12:45
Les aiguillons 12:45
Eglise 12:52

The actual map will look something like this:

None



```
        STOP 2  •
                |
                |
        STOP 1  •
                |
                |
         STARTING POINT
```

This is only a simplified illustration.

In the actual application, the route will follow real roads instead of a straight line.

Important: We will not connect stops using a simple straight line. We will use road directions to create a realistic route.

STEP 3: Choose the Technology

For this feature, I recommend:

Technology	Purpose
Flutter	Application interface
Google Maps Flutter	Display the map
Firebase Firestore	Store bus location and route information
Firebase Authentication	Identify drivers and passengers
GPS	Get the bus's current location
Google Maps Directions API	Get the road route

Google Maps Routes API	Calculate travel time and route information

We will use Google Maps for the map and Firebase for live location synchronization.

The basic architecture

```
None
          DRIVER PHONE
               |
               |
             GPS
               |
               |
         FLUTTER DRIVER APP
               |
               |
        FIREBASE FIRESTORE
               |
               |
       LIVE LOCATION UPDATES
               |
               |
        STUDENT / TEACHER APP
               |
               |
         GOOGLE MAPS WIDGET
               |
               |
       BUS MOVES ON THE MAP
```

STEP 4: Understand How Live Tracking Works

This is the most important concept.

The driver's phone has GPS.

GPS provides:

- Latitude.
- Longitude.
- Location accuracy.
- Timestamp.

For example:

```JSON
{
  "latitude": 12.9716,
  "longitude": 77.5946
}
```

These coordinates represent the bus's current location.

When the bus moves, the coordinates change.

Example:

Time	Latitude	Longitude
7:30 AM	12.9716	77.5946
7:31 AM	12.9720	77.5950
7:32 AM	12.9725	77.5954
7:33 AM	12.9730	77.5960

The driver application sends the updated coordinates to Firebase.

The student application receives the coordinates.

The bus marker moves accordingly.

This is live tracking.

STEP 5: Create the Firebase Database

Open your Firebase project.

Go to:

Firebase Console → Firestore Database

Create a collection:

```
None
bus_tracking
```

Inside it, create 7 documents.

```
None
bus_tracking
    |
    |-- BUS001
    |-- BUS002
    |-- BUS003
    |-- BUS004
    |-- BUS005
```

```
    |-- BUS006
    |-- BUS007
```

Each document represents one bus.

Example: BUS001

JSON

```
{
  "busId": "BUS001",
  "busNumber": "KA01AB1234",
  "routeId": "ROUTE001",
  "driverId": "DRIVER001",

  "latitude": 12.9716,
  "longitude": 77.5946,

  "status": "NOT_STARTED",
  "trackingEnabled": false,

  "lastUpdated": null
}
```

Explanation

`busId` identifies the bus.

`routeId` identifies the route.

`driverId` identifies the driver.

`latitude` and `longitude` store the current GPS location.

`status` tells whether the bus has started.

`trackingEnabled` tells whether live tracking is active.

`lastUpdated` stores the last location update time.

Repeat this for all 7 buses.

STEP 6: Create the Routes Collection

Now create another collection:

```
None
routes
```

Each bus has a separate route.

Example:

```
None
routes
    |
    |-- ROUTE001
    |-- ROUTE002
    |-- ROUTE003
    |-- ROUTE004
    |-- ROUTE005
    |-- ROUTE006
```

```
    |-- ROUTE007
```

Each route contains 10 stops.

Example: ROUTE001

JSON

```
{
  "routeId": "ROUTE001",
  "routeName": "Route 1",
  "busId": "BUS001",
  "destination": "College Campus",

  "stops": [
    {
      "stopId": "STOP01",
      "stopName": "Stop 1",
      "latitude": 12.9500,
      "longitude": 77.5500,
      "sequence": 1
    },
    {
      "stopId": "STOP02",
      "stopName": "Stop 2",
      "latitude": 12.9550,
      "longitude": 77.5550,
      "sequence": 2
    },
    {
      "stopId": "STOP03",
```

```
      "stopName": "Stop 3",
      "latitude": 12.9600,
      "longitude": 77.5600,
      "sequence": 3
    }
  ]
}
```

This example contains only 3 stops to demonstrate the structure.

Your actual route will contain 10 stops.

Why do we need stop coordinates?

Because the map needs to know where each stop is located.

For every stop, you need:

- Stop name.
- Latitude.
- Longitude.
- Stop sequence.

The sequence determines the order of the stops.

Example:

```
None
STOP01 → STOP02 → STOP03 → STOP04 → ... → STOP10 → COLLEGE
```

The bus will follow this sequence.

STEP 7: Display the Map Inside Flutter

Now we start the actual Flutter implementation.

First, install the required packages.

Open your Flutter project terminal.

Run:

```shell
flutter pub add google_maps_flutter
```

Then:

```shell
flutter pub add geolocator
```

Then:

```shell
flutter pub add cloud_firestore
```

Then:

```shell
flutter pub add firebase_core
```

These packages provide:

- Google Maps.
- GPS location.

- Firestore database access.
- Firebase initialization.

Next, configure Google Maps for Android.

Open:

```
None
android/app/src/main/AndroidManifest.xml
```

Add the location permissions:

```
XML
<uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION"/>

<uses-permission
    android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

Then configure your Google Maps API key according to the Google Maps Flutter setup instructions.

Official documentation:

- Google Maps for Flutter
- Geolocator Flutter package
- Firebase Flutter setup

STEP 8: Create the Map Screen

Create a Flutter file:

```
None
bus_tracking_screen.dart
```

Import:

```
Dart (unsupported)
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
```

Create the screen:

```
Dart (unsupported)
class BusTrackingScreen extends StatefulWidget {
  const BusTrackingScreen({super.key});

  @override
  State<BusTrackingScreen> createState() =>
      _BusTrackingScreenState();
}
```

Now create the state:

```
Dart (unsupported)
class _BusTrackingScreenState
    extends State<BusTrackingScreen> {

  GoogleMapController? mapController;

  final LatLng collegeLocation =
      LatLng(12.9716, 77.5946);
```

```
  @override
  Widget build(BuildContext context) {

    return Scaffold(

      appBar: AppBar(
        title: const Text("Live Bus Tracking"),
      ),

      body: GoogleMap(

        initialCameraPosition: CameraPosition(
          target: collegeLocation,
          zoom: 12,
        ),

        onMapCreated: (controller) {
          mapController = controller;
        },

        markers: {},

        polylines: {},

      ),

    );
  }
}
```

What this code does

It creates a Google Map inside your Flutter application.

Currently:

- The map is displayed.
- The college location is the initial camera position.
- No bus marker is displayed yet.
- No route is displayed yet.

We will add those next.

STEP 9: Display the 10 Stops as Circle Markers

This is exactly the feature you requested.

Each stop should appear as a circle.

For example:

```
None
• Stop 1
• Stop 2
• Stop 3
• Stop 4
• Stop 5
• Stop 6
• Stop 7
• Stop 8
• Stop 9
```

```
• Stop 10
```

We will use Google Maps markers.

Create a list of stops:

```
Dart (unsupported)
final List<LatLng> stops = [
  LatLng(12.9500, 77.5500),
  LatLng(12.9550, 77.5550),
  LatLng(12.9600, 77.5600),
  LatLng(12.9650, 77.5650),
  LatLng(12.9700, 77.5700),
  LatLng(12.9750, 77.5750),
  LatLng(12.9800, 77.5800),
  LatLng(12.9850, 77.5850),
  LatLng(12.9900, 77.5900),
  LatLng(12.9950, 77.5950),
];
```

These are example coordinates, not actual college bus stops.

Now create the markers:

Dart (unsupported)

```
Set<Marker> createStopMarkers() {

  Set<Marker> markers = {};

  for (int i = 0; i < stops.length; i++) {

    markers.add(

      Marker(

        markerId: MarkerId("STOP$i"),

        position: stops[i],

        infoWindow: InfoWindow(
          title: "Stop ${i + 1}",
        ),

      ),

    );

  }

  return markers;

}
```

What happens?

The application loops through the 10 stops.

Each stop becomes a marker.

When users tap the marker, they see its stop name.

Later, we will load these stops from Firestore instead of hardcoding them.

STEP 10: Draw the Route Line

You said:

> The map should be shown like a line graph, each stop as a point on the line.

We can implement this using a Google Maps polyline.

But remember:

A normal line between coordinates will not follow roads.

For example:

```
None
STOP1 ●
      |
STOP2 ●
      |
STOP3 ●
      |
STOP4 ●
```

This is a straight-line representation.

For actual bus transportation, we want:

```
None
STOP1 ●──────┐
             |
STOP2 ●──────┘
      |
      └──────STOP3 ●
                   |
             STOP4 ●
```

The real route should follow roads.

For now, create a polyline

```dart
Polyline createRoute() {

  return Polyline(

    polylineId: const PolylineId("BUS_ROUTE"),

    points: stops,

    width: 5,

  );

}
```

Then update your map:

```
Dart (unsupported)
GoogleMap(

  initialCameraPosition: CameraPosition(
    target: collegeLocation,
    zoom: 12,
  ),

  markers: createStopMarkers(),

  polylines: {
    createRoute(),
  },

)
```

Now the map displays:

- 10 stop markers.
- A line connecting the stops.

Important: The sample line connects the coordinates directly. Later, we will replace it with a road-following route generated from the Directions API.

STEP 11: Display the Bus Emoji 🚌

This is one of the most important parts.

You want:

- A bus emoji when the bus is moving.
- A plain circle when the bus is stopped at a location.

We can create a custom bus marker.

Google Maps Flutter supports custom marker icons.

For a simple implementation, we can create a bus icon image and use it as the marker.

The marker will have:

```
None
🚌
```

The bus marker will represent the live GPS location.

Bus marker logic

```
None
IF BUS IS MOVING

    DISPLAY 🚌

ELSE

    DISPLAY ●
```

We need to determine whether the bus is moving.

The driver application can send:

- Current GPS coordinates.
- Previous GPS coordinates.
- Speed.
- Timestamp.

If the bus speed is above a threshold, we can consider it moving.

For example:

```
Speed > 3 km/h

BUS MOVING

Display 🚌
```

If:

```
Speed <= 3 km/h

BUS STOPPED

Display ●
```

However, we should not rely only on speed.

GPS accuracy can fluctuate.

We should also check whether the bus is near a scheduled stop and whether the driver has marked the bus as stopped.

Important implementation detail

Google Maps Flutter does not directly support displaying a Unicode emoji as a marker in the same way as ordinary text.

We should create a custom marker image from the bus emoji or use a small bus icon image.

This will make the bus marker visible and consistent across Android devices.

STEP 12: Update the Bus Location in Real Time

Now we connect the driver GPS to Firebase.

The driver application obtains the GPS coordinates.

Example:

```dart
Position position =
    await Geolocator.getCurrentPosition();
```

Then:

```dart
double latitude = position.latitude;

double longitude = position.longitude;
```

The driver application sends those coordinates to Firestore.

Example:

```dart
FirebaseFirestore.instance
    .collection("bus_tracking")
    .doc("BUS001")
    .update({
```

```
  "latitude": latitude,

  "longitude": longitude,

  "lastUpdated":
      FieldValue.serverTimestamp(),

});
```

Every time the driver location changes, the database updates.

How often should the location update?

For your application, I recommend:

Bus condition	Update interval
Moving	Every 5–10 seconds
Stopped	Every 20–30 seconds
Journey completed	Stop tracking

These are starting values.

We can later optimize based on GPS accuracy, battery consumption, and network connectivity.

STEP 13: Make the Bus Move on the Student Map

Now we listen to Firestore.

The student application will receive the latest bus coordinates.

Example:

```dart
FirebaseFirestore.instance
    .collection("bus_tracking")
    .doc("BUS001")
    .snapshots()
```

This gives us a real-time stream.

Whenever the bus location changes:

```
BUS001 LOCATION UPDATED
          |
          |
       FIRESTORE
          |
          |
      STUDENT APP
          |
          |
      UPDATE MARKER
          |
          |
       🚌 MOVES
```

Bus marker

The marker should use the latest coordinates.

Example:

```
Dart (unsupported)
Marker(

  markerId: const MarkerId("BUS001"),

  position: LatLng(
    busLatitude,
    busLongitude,
  ),

  infoWindow: const InfoWindow(
    title: "College Bus",
  ),

)
```

Every time the coordinates change, we update the marker position.

This is the main live tracking functionality.

STEP 14: Display Bus Delay

Now we implement the delay feature.

You want:

> If there is any bus delay, show where the delay is and for how many minutes.

This is a very important feature.

But we must distinguish between:

1. A bus being stopped.
2. A bus being delayed.

A bus stopping at a scheduled pickup point is normal.

A bus stopping unexpectedly for a long time may indicate a delay.

Example

Suppose:

Stop	Scheduled arrival
Stop 1	7:00 AM
Stop 2	7:10 AM
Stop 3	7:20 AM
Stop 4	7:30 AM
College	8:00 AM

The bus reaches Stop 3 at 7:28 AM.

Scheduled arrival:

7:20 AM

Actual arrival:

7:28 AM

Delay:

8 minutes

The application should display:

```
None
⚠ BUS DELAY

Location:
Stop 3

Delay:
8 minutes
```

The bus marker can also show a delay indicator.

STEP 15: Calculate the Delay

Each stop needs a scheduled arrival time.

Example:

```
JSON
{
  "stopId": "STOP03",
  "stopName": "Stop 3",
  "sequence": 3,
  "scheduledArrival": "07:20"
}
```

When the bus arrives, the driver application records the actual arrival time.

Example:

```JSON
{
  "stopId": "STOP03",
  "actualArrival": "07:28"
}
```

Calculate:

```None
Delay = Actual Arrival - Scheduled Arrival
```

Example:

```None
Actual Arrival = 07:28

Scheduled Arrival = 07:20

Delay = 8 minutes
```

The application displays the delay.

Important

We should calculate delay using the scheduled time and actual arrival time.

We should not simply assume that the bus is delayed because it has stopped moving.

STEP 16: Show Where the Bus Is Delayed

Suppose Bus 1 is delayed between Stop 4 and Stop 5.

The application should display:

```
None
BUS 1

Current Location:
Between Stop 4 and Stop 5

Status:
DELAYED

Delay:
12 minutes
```

On the map:

```
None
STOP 4 ●
        |
        |
     🚌 ⚠
        |
        |
STOP 5 ●
```

The delay marker should be located at the bus's current GPS position.

Delay information

We should display:

- Bus number.
- Current location.
- Nearest stop.
- Delay duration.
- Delay reason, if available.
- Last updated time.

Example:

```
None
BUS 1 DELAY

Current Location:
Near Stop 5

Delay:
12 minutes

Reason:
Traffic congestion

Last Updated:
7:42 AM
```

The driver can optionally select the delay reason.

Possible reasons:

- Traffic congestion.
- Road closure.

- Mechanical issue.
- Passenger pickup.
- Other.

STEP 17: Calculate Delay Automatically

We can calculate delays using scheduled and actual arrival times.

For example:

```
None
Scheduled Arrival: 7:30 AM

Current Time: 7:42 AM

Bus has not reached the stop.

Estimated Delay: 12 minutes
```

But this is only correct if the bus is expected to have reached the stop by 7:30 AM.

If the bus is still moving toward the stop and its estimated arrival is 7:35 AM, then the delay is only 5 minutes.

Therefore, the better approach is:

Delay = Expected Arrival Time − Scheduled Arrival Time

We should use the estimated arrival time from the routing service.

For example:

```
None
Scheduled Arrival: 7:30 AM

Estimated Arrival: 7:42 AM

Delay: 12 minutes
```

This gives a more useful delay estimate.

STEP 18: How the 7 Routes Will Work

We will create 7 route documents.

Each route has:

- 10 stops.
- Starting location.
- College destination.
- Scheduled arrival times.
- Assigned bus.

Example:

```
None
ROUTE001
    |
    |-- STOP01
    |-- STOP02
    |-- STOP03
    |-- STOP04
```

```
    |-- STOP05
    |-- STOP06
    |-- STOP07
    |-- STOP08
    |-- STOP09
    |-- STOP10
    |
    |-- COLLEGE
```

The same structure applies to all 7 routes.

Important

We will not create one tracking system per route.

We will create one reusable tracking system.

The application will load the correct bus and route from Firestore.

For example:

```
None
Student assigned to ROUTE001

    |

BUS001

    |

Display BUS001 tracking
```

A teacher assigned to ROUTE005 will see BUS005.

STEP 19: Final Map Design

Your final map should contain the following.

Map

- Road-following route line.
- 10 stop markers.
- College destination marker.
- Bus emoji marker.
- Delay indicator.
- Current bus location.

Information panel

```
None
--------------------------------

BUS 1

Route:
Route 1

Status:
ON THE WAY

Current Location:
Near Stop 6
```

```
Next Stop:
Stop 7

Estimated Arrival:
7:45 AM

Delay:
5 minutes

--------------------------------

[ LIVE TRACKING ]

--------------------------------
```

When the bus is delayed:

```
None
--------------------------------

⚠ BUS DELAY

Bus:
BUS 1

Location:
Near Stop 6

Delay:
```

```
12 minutes

Reason:
Traffic congestion

--------------------------------
```

STEP 20: Important Things You Must Know

1. Google Maps location sharing

Your existing Google Maps sharing link can be saved as a backup.

However, we cannot directly use that link as a reliable source of GPS coordinates for the embedded map.

The driver application must provide the GPS coordinates.

2. GPS tracking

The driver's phone must have:

- Location enabled.
- Required permissions.
- Internet connectivity.

If the phone loses internet, the location cannot reach Firebase until connectivity returns.

3. Background tracking

For daily bus tracking, the driver application must continue updating location when the screen is locked or the application is minimized.

This requires proper background location configuration.

4. Delay calculation

A bus stopping at a scheduled stop is not automatically delayed.

We must compare scheduled and actual arrival times.

5. Route accuracy

The route should follow real roads.

A simple polyline connecting 10 coordinates is not enough for accurate bus navigation.

We will use a routing service to generate the road-following route.

Your Implementation Roadmap

Build the feature in this order.

Step	Task
1	Create Firebase project
2	Configure Firestore
3	Create 7 bus documents
4	Create 7 route documents
5	Add 10 stops per route
6	Configure Google Maps
7	Display map

8	Display stop markers
9	Draw route line
10	Create bus marker
11	Get driver GPS
12	Send GPS to Firestore
13	Listen to live bus location
14	Move bus marker
15	Calculate estimated arrival
16	Calculate delays
17	Display delay location
18	Test all 7 routes

My Recommendation: Build One Bus First

Do not start by implementing all 7 buses.

Start with:

Bus 1 → Route 1 → 10 Stops → College

Make sure:

- GPS works.
- Firebase updates work.
- The bus marker moves.
- The route displays correctly.

- Stops appear correctly.
- Delay calculation works.

Then reuse the same code for the remaining 6 buses.

This will make your implementation much easier.

Before we move to actual coding

There is one important thing to decide about the delay system:

How should the application detect a bus delay?

A. Automatic: The application compares scheduled arrival time with the bus's estimated arrival time and automatically displays delays.

B. Driver-based: The driver manually marks the bus as delayed and selects the reason.

C. Both: The application automatically detects delays, and the driver can also report the reason.

I recommend C. Automatic detection gives passengers useful information, while the driver can explain the actual reason for the delay.